```
#!/usr/bin/env python3
"""
Command-Line To-Do List App
A beginner-friendly Python project
demonstrating:
- File I/O (saving/loading JSON)
- Classes and objects (OOP)
- argparse for command-line arguments
- Basic error handling

Usage:
    python todo.py add "Buy groceries"
    python todo.py list
    python todo.py done 1
    python todo.py remove 1
"""

import json
import argparse
import os
from datetime import datetime

DATA_FILE = "tasks.json"


class Task:
    def __init__(self, id, description,
done=False, created=None):
        self.id = id
        self.description = description
```

```
        self.done = done
        self.created = created or
datetime.now().strftime("%Y-%m-%d %H:%M")

    def to_dict(self):
        return {
            "id": self.id,
            "description":
self.description,
            "done": self.done,
            "created": self.created,
        }

    @staticmethod
    def from_dict(data):
        return Task(data["id"],
data["description"], data["done"],
data["created"])

    def __str__(self):
        status = "✔" if self.done else " "
        return f"[{status}] {self.id}.
{self.description} (added {self.created})"


class TodoList:
    def __init__(self, filepath=DATA_FILE):
        self.filepath = filepath
        self.tasks = self._load()

    def _load(self):
        if not
os.path.exists(self.filepath):
            return []
```

```
        try:
            with open(self.filepath, "r")
as f:
                data = json.load(f)
                return [Task.from_dict(t)
for t in data]
        except (json.JSONDecodeError,
KeyError):
            print("Warning: tasks file was
corrupted. Starting fresh.")
            return []

    def _save(self):
        with open(self.filepath, "w") as f:
            json.dump([t.to_dict() for t in
self.tasks], f, indent=2)

    def _next_id(self):
        return max((t.id for t in
self.tasks), default=0) + 1

    def add(self, description):
        task = Task(self._next_id(),
description)
        self.tasks.append(task)
        self._save()
        print(f"Added task {task.id}:
{description}")

    def list(self):
        if not self.tasks:
            print("No tasks yet. Add one
with: python todo.py add \"Your task\"")
            return
```

```
        for task in self.tasks:
            print(task)

    def complete(self, task_id):
        for task in self.tasks:
            if task.id == task_id:
                task.done = True
                self._save()
                print(f"Marked task
{task_id} as done.")
                return
        print(f"No task found with id
{task_id}.")

    def remove(self, task_id):
        before = len(self.tasks)
        self.tasks = [t for t in self.tasks
if t.id != task_id]
        if len(self.tasks) < before:
            self._save()
            print(f"Removed task
{task_id}.")
        else:
            print(f"No task found with id
{task_id}.")


def main():
    parser =
argparse.ArgumentParser(description="A
simple command-line to-do list.")
    subparsers =
parser.add_subparsers(dest="command")
```

```
    add_parser =
subparsers.add_parser("add", help="Add a
new task")
    add_parser.add_argument("description",
type=str, help="Task description")

    subparsers.add_parser("list",
help="List all tasks")

    done_parser =
subparsers.add_parser("done", help="Mark a
task as complete")
    done_parser.add_argument("id",
type=int, help="Task ID")

    remove_parser =
subparsers.add_parser("remove",
help="Remove a task")
    remove_parser.add_argument("id",
type=int, help="Task ID")

    args = parser.parse_args()
    todo = TodoList()

    if args.command == "add":
        todo.add(args.description)
    elif args.command == "list":
        todo.list()
    elif args.command == "done":
        todo.complete(args.id)
    elif args.command == "remove":
        todo.remove(args.id)
    else:
        parser.print_help()
```

```
if __name__ == "__main__":
    main()
```